

Technical Art Documentation: Entropy Loop

Chapter 1 : Introduction

1.1. Introduction

This document explains about the technical art pipeline used during the development of Entropy Loop. From how 3D assets are created and processed to animated and integrated into Unity.

The project's artistic direction is documented separately in the Art Bible. This document focuses exclusively on the technical workflow and production processes that support the implementation of the game's visual content.

Development Environment

- Engine: Unity 6
- 3D Software: Blender
- AI Modeling Tool: Meshy.ai
- Animation System: Unity Animation Rigging
- Version Control: *Git with GitHub repository*
- Team Size: 6 members

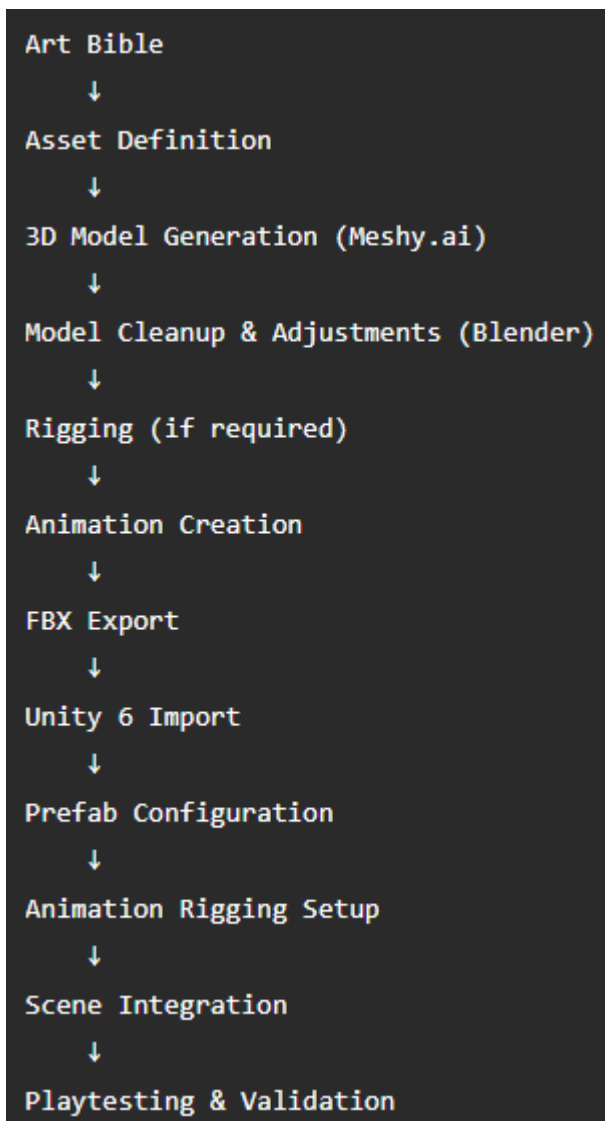
Chapter 2 : Production Pipeline

2.1. Pipeline Overview

The production pipeline shows how the workflow used to transform a card concept into a fully game model asset. Using a clear pipeline helps to get consistency between team members, simplifies asset management, and reduces production errors throughout the development process.

The pipeline has been designed to be alterable and adaptable as new content is produced.

The overall workflow is illustrated below:



2.2. Software and Tools

Entropy Loop relies on several software solutions, each used for a specific step of the production pipeline.

Software	Purpose
Meshy.ai	Ai-assisted generation to transform concept art into base 3D Models
Blender	Model editing, rigging, animation and FBX export
Unity (6.0)	Game engine, gameplay implementation, asset integration and testing

2.3. Production Workflow

The production process follows these steps:

1. Define the asset requirements based on the Art Bible and gameplay needs.
2. Generate an initial 3D model using Meshy.ai.
3. Import the generated model into Blender for cleanup and structural adjustments.
4. Create the character skeleton.
5. Produce the required animations.
6. Export the asset as an FBX file.
7. Import the asset into Unity 6.
8. Configure prefabs and Animation Rigging.
9. Test the asset in-game and validate its functionality.

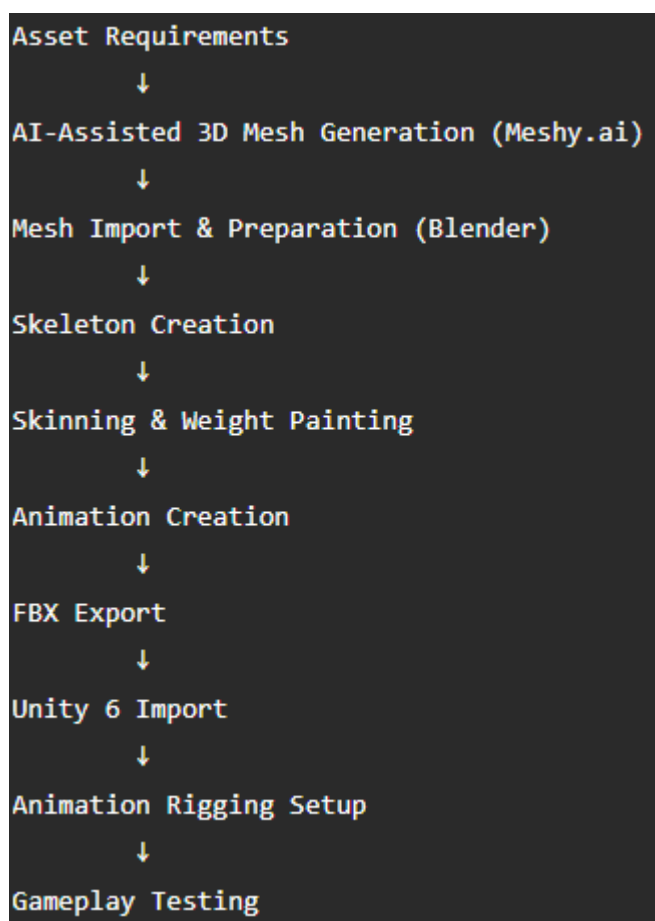
This iterative workflow allows assets to be upgraded continuously during development.

Chapter 3 : 3D Asset Creation

3.1. 3D Asset Creation Workflow

The 3D asset creation pipeline is made to make the production of game-ready models faster while controlling the final result. The workflow fuses AI-assisted generation with traditional 3D production methods to create assets suitable for integration into Unity 6.

The creation process is divided into several stages:



Each stage checks that the asset meets both artistic requirements and technical constraints.

3.2. AI-Assisted Model Generation

Purpose

Meshy.ai is used as an AI-assisted modeling tool to accelerate the early stages of asset creation.

Instead of creating every model from scratch, the team generates base 3D meshes that serve as a starting base for further preparation and integration.

This approach allows the team to:

- Quickly prototype visual ideas.
- Produce playable assets earlier in development.
- Focus production time on gameplay functionality, rigging, and animation.

The generated meshes are not considered final assets. They require additional preparation before being used in the game.

Generation Process

The generation workflow consists of:

1. Defining the required asset based on gameplay needs and the Art Bible.
2. Creating a description of the desired model.
3. Generating multiple mesh variations.
4. Selecting a suitable result.
5. Importing the mesh into Blender for preparation.

3.3. Blender Preparation Pipeline

After generation, the models are imported into Blender.

At the current stage of development, Blender is mainly used for character preparation rather than complete asset production.

Model Preparation

Before rigging, the mesh is checked for:

- Correct scale.
- Object orientation.
- Mesh accuracy.
- Compatibility with the rigging process.

Basic corrections may be applied when necessary to improve deformation quality during animation.

Skeleton Creation

Characters require a custom armature before they can be animated.

The rigging process includes:

- Creating the bone structure.
- Placing bones according to the character proportions.
- Preparing the mesh for deformation.
- Assigning weights to the skeleton.

The goal is to create a functional rig that allows controlled character movement inside Unity.

Animation Creation

Animations are created directly in Blender.

The process includes:

- Creating keyframes.
- Adjusting poses.
- Testing movement quality.
- Exporting animation data with the FBX file.

The animations are designed according to gameplay requirements, such as:

- Idle animations.
- Movement animations.
- Combat actions.

3.4. FBX Export Pipeline

Once the model and animations are prepared, they are exported from Blender using the FBX format.

The export process ensures compatibility between Blender and Unity 6.

The exported file contains:

- 3D mesh data.
- Skeleton information.
- Animation clips.

Before importing into Unity, the following elements are verified:

- Correct orientation.
- Applied transformations.
- Animation compatibility.
- File naming consistency.

3.5. Current Limitations

The current production pipeline is optimized for fast development and gameplay iteration. However, some steps are pinpoint for later production phases.

Current limitations:

- Final textures are not implemented yet.
- Materials are still in development.
- Generated meshes may require additional optimization.
- Visual consistency between assets requires further validation.

Future improvements:

- Establish a complete texturing workflow.
- Define final material standards.
- Optimize assets for final performance targets.
- Create a complete asset validation process.

This approach allows the team to prioritize gameplay development while maintaining a clear path toward final visual production.

Chapter 4 : Character Rigging & Animation

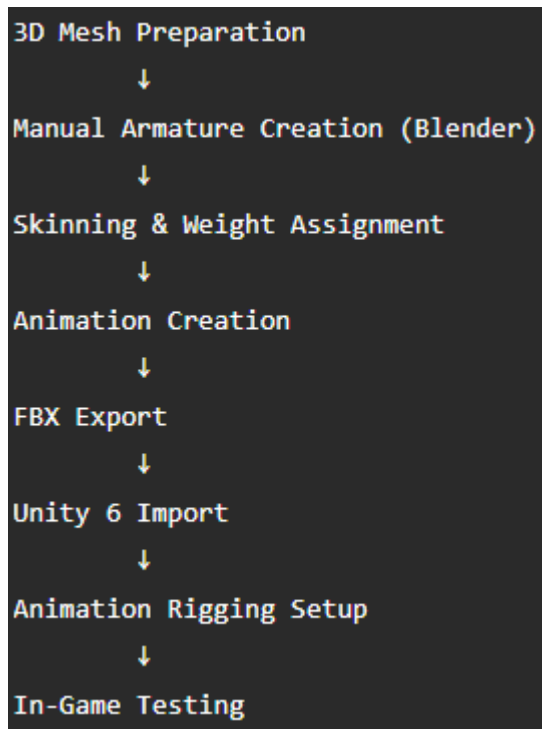
4.1. Overview

Character rigging and animation are essential steps in making static 3D meshes into functional game characters.

The character preparation is performed manually in Blender. Each generated mesh is adapted with a custom skeleton before animations are created and integrated into Unity 6.

The current workflow focuses on creating lightweight and controllable character rigs suitable for real-time gameplay.

The workflow is structured as follows:



4.2 Manual Rigging Process

Armature Creation

Without used fully automated rigging solutions, the project uses manually created armatures in Blender.

This approach provides better control over:

- Bones placements.
- Character proportions.
- Animation requirements.
- Deformation quality.

The skeleton is created according to the structure and needs of each character.

The main steps are:

1. Creating the bone hierarchy.
2. Defining parent-child relationships between bones.
3. Positioning bones according to the character anatomy.
4. Preparing the armature for skinning.

Skinning and Weight Assignment

After the armature is created, the mesh is linked to the skeleton through weight painting.

This step determines how each part of the model reacts to bone movements.

The process includes:

- Assigning mesh vertices to bones.
- Adjusting weight influence.
- Testing deformation quality.
- Correcting problematic areas.

The goal is to achieve natural deformation while keeping the rig simple enough for real-time usage.

4.3. Animation Production

Animations are created manually in Blender according to gameplay requirements.

The hand-made animation workflow allows the team to control:

- Character personality.
- Movement timing.
- Visual feedback.
- Gameplay readability.

The animation process includes:

1. Creating key poses.
2. Refining transitions between poses.
3. Adjusting timing and movement intensity.
4. Testing animations with the character rig.
5. Exporting animation clips.

Examples of animation types used in the project include:

- Idle animations.
- Movement animations.
- Combat animations.

4.4. Unity Animation Integration

After export, character assets are imported into Unity 6.

The integration process includes:

- Importing the FBX file.
- Configuring the character model.
- Setting up animation clips.
- Connecting animations to gameplay systems.

The Unity Animation Rigging package is used to provide additional control over character movement and improve the flexibility of the animation system.

This allows the team to adjust character behavior directly inside the engine when required.

4.5. Animation Validation

Before validating an animation ready for integration, they are tested according to several criteria:

Validation	Objective
Visual quality	Ensure movements match the artistic direction
Gameplay readability	Make actions understandable during gameplay
Technical compatibility	Verify correct import into Unity
Performance	Ensure animations remain suitable for real-time execution

4.6. Current Limitations

The current animation pipeline allows the team to quickly create functional characters for gameplay testing.

However, improvements are planned:

- Refinement of animation quality.
- Creation of additional animation variations.
- Standardization of rig structures.
- Optimization of character assets for final production.
- Implementation of final visual polish.

The current system provides a flexible foundation for future character development.

Chapter 5 : Unity Integration

5.1. Unity Environment

Unity 6 is used as the main game engine for asset integration, gameplay implementation, and testing.

The project currently uses the standard Unity rendering pipeline. This choice allows the team to maintain a n easy and flexible rendering setup while focusing on gameplay development and asset integration during the beta phase.

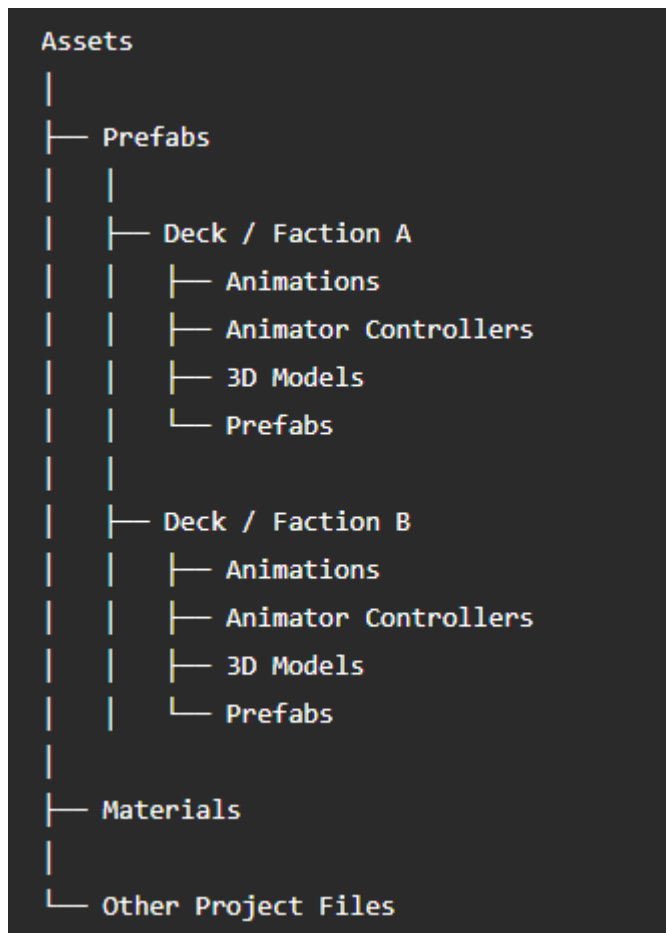
The Unity pipeline is responsible for:

- Importing 3D assets.
- Managing prefabs.
- Configuring animations.
- Applying materials.
- Testing assets directly in gameplay conditions.

5.2. Project Organization

To maintain a clear and scalable structure, assets are organized according to their function and their gameplay purpose.

The current project structure follows this organization:



5.3. Model Import Workflow

After being finished in Blender, models are exported as FBX files and imported into Unity.

The integration workflow follows these steps:

1. Export the model and animation data from Blender.
2. Import the FBX file into the corresponding Unity folder.
3. Verify model orientation and scale.
4. Configure animation settings.
5. Create or update the corresponding prefab.
6. Test the asset directly in the game environment.

The import process ensures that the model, skeleton, and animations remain correctly connected between Blender and Unity.

5.4. Prefab System

Prefabs are used as the main method for managing reusable game objects.

Each unit is converted into a prefab containing the necessary components for gameplay integration.

A typical character prefab includes:

- 3D Model component.
- Animator component.
- Animation Controller reference.
- Materials.
- Gameplay-related components.

Using prefabs allows the team to modify an asset once and automatically apply changes wherever it is used.

5.5. Animation Management

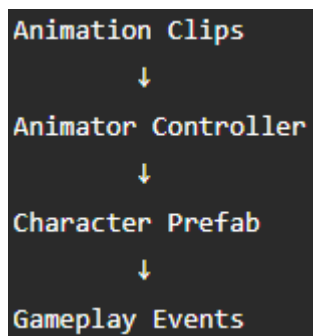
Animations are separated from the models and stored inside dedicated animation folders.

Each unit contains:

- Animation clips.
- Animator Controller.
- Model files.

The Animator Controller is responsible for managing animation states and transitions.

A typical animation workflow is:



5.6. Material Management and Visual Differentiation

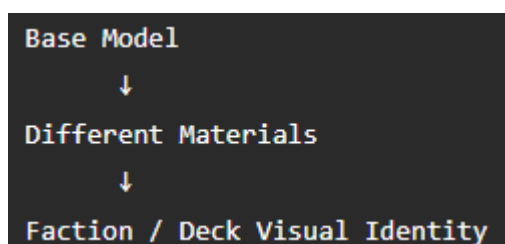
At the current development stage, textures are not yet finalized.

Instead, Unity materials are mainly used to create visual variations between units and factions.

Different materials allow the team to:

- Quickly identify different unit categories.
- Test gameplay readability.
- Create temporary visual differentiation before final texturing.

The current material workflow is based on color variation:



5.7. Current Integration Limitations

The current Unity integration pipeline is functional for prototyping and gameplay testing.

However, future improvements are planned:

- Creation of final textures.
- Replacement of temporary material variations.
- Improvement of material consistency.
- Implementation of final visual effects.
- Optimization of asset loading and performance.

The current structure provides a stable foundation for the final production phase.

Chapter 6 : Asset Standards

6.1. Naming Convention

To maintain consistency and simplify asset management, each asset follows a naming convention based on the unit or character name.

The main objective of this system is to keep all related resources grouped together and easily identifiable during production.

Each unit has a shared naming identifier, which is used across its different assets.

Example:

KeronasPrefab.prefab
KeronasIdle.anim
KeronasAttack.anim
KeronasAnimator.controller

Using a common identifier allows team members to quickly associate models, animations, and controllers with the correct unit.

6.2. Asset Naming Rules

Prefabs

Prefab files use the following structure:

[UnitName]Prefab.prefab

Example:

KeronasPrefab.prefab

Prefabs represent the complete Unity object and contain the required components for gameplay integration.

Animations

Animation clips use the unit name followed by the animation type.

Structure:

[UnitName][AnimationType].anim

Examples:

KeronasIdle.anim

KeronasAttack.anim

This naming system makes animation management easier when configuring Animator Controllers.

Animator Controllers

Animator Controllers follow the same identification system:

[UnitName]Animator.controller

Example:

KeronasAnimator.controller

Each controller manages the animation states and transitions of a specific unit.

Materials

Materials do not currently follow a finalized naming convention.

During the beta phase, materials are mainly used to create visual variations between different factions or decks.

A future naming system will be implemented when the final texturing and material pipeline is established.

Suggested future structure:

[UnitName]Material_[Variation]

Example:

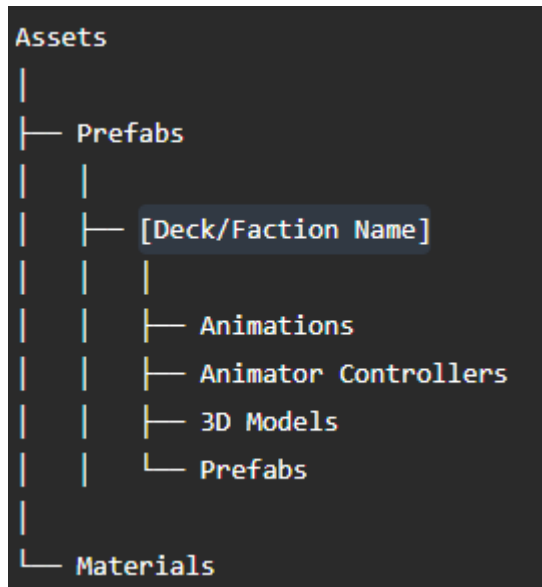
KeronasMaterial_Red

KeronasMaterial_Blue

6.3. Folder Organization Rules

Assets are organized according to their gameplay category, mainly through faction/deck separation.

Current structure:



This structure allows each unit group to keep all required resources together.

Benefits:

- Faster asset retrieval.
- Easier collaboration between team members.
- Reduced risk of file duplication.
- Better scalability when adding new units.

6.4. File Format Standards

The project uses the following file formats:

Asset Type	Format	Purpose
3D Models	FBX	Blender to Unity export
Animations	FBX / Unity Animation Clips	Character movement
Prefabs	Unity Prefab	Playable game objects
Materials	Unity Material	Visual variations

6.5. Asset Validation Checklist

Before integrating an asset into the game, the following checks are performed:

Validation	Objective
Naming consistency	Ensure correct identification
Correct folder placement	Maintain project organization
Prefab functionality	Verify gameplay integration
Animation assignment	Confirm Animator setup
Visual readability	Ensure unit differentiation
In-game testing	Validate final behavior